2024

Глава 5 Динамические структуры данных. Списки

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

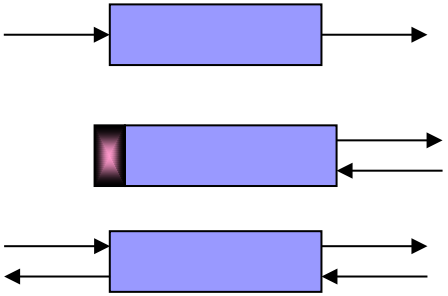
5.1 Классификация структур данных

- Структуры данных
  - Последовательные
    - Массив (вектор, матрица)
    - Строка
    - Структура
  - Древовидные
    - Бинарные деревья
    - Сортированные Бинарные деревья
  - Сетевые

Динамическими называют структуры, у которых могут меняться конфигурация, размеры и/или состав. Обычно размещаются в динамической памяти ("куче"). Могут реализовываться с использованием массивов и списков.

Классические динамические последовательные структуры:

1. Очередь – структура данных, реализующая добавление – в конец, а удаление – из начала.
2. Стек – структура данных, реализующая добавление и удаление с одной стороны.
3. Дек – структура данных, реализующая добавление и удаление с двух сторон.



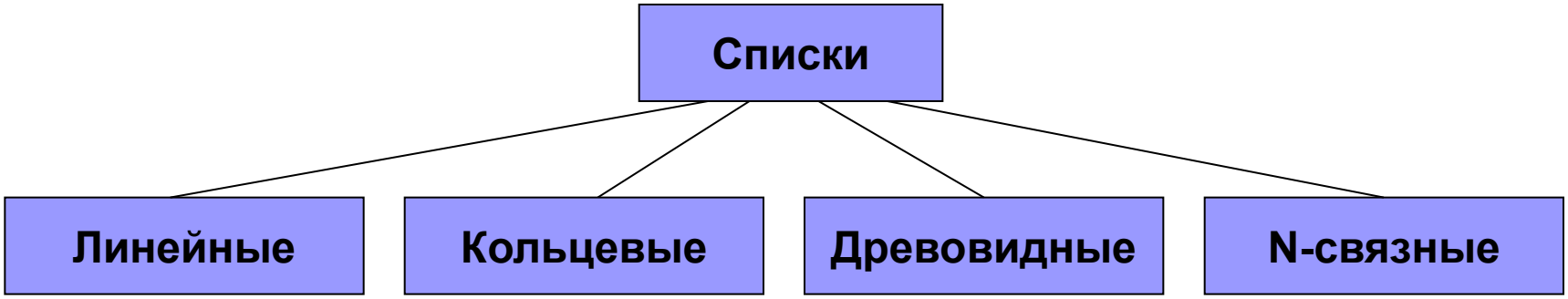
5.2 Списки

Список – способ организации данных, предполагающий использование указателей для определения следующего элемента.

Элемент списка состоит из двух частей: *информационной* и *адресной*.

Информационная часть содержит поля данных.

Адресная – включает от одного до n указателей, содержащих адреса следующих элементов. Количество связей, между соседними элементами списка определяет его связность: односвязные, двусвязные, n-связные.

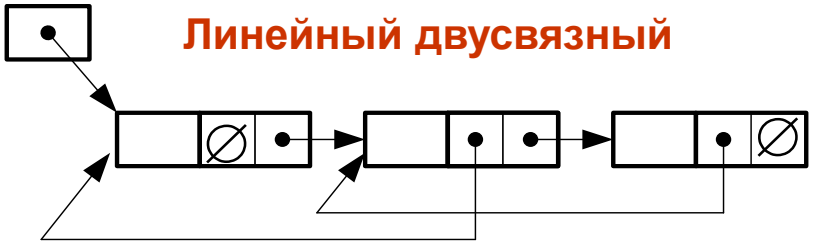


Виды списков

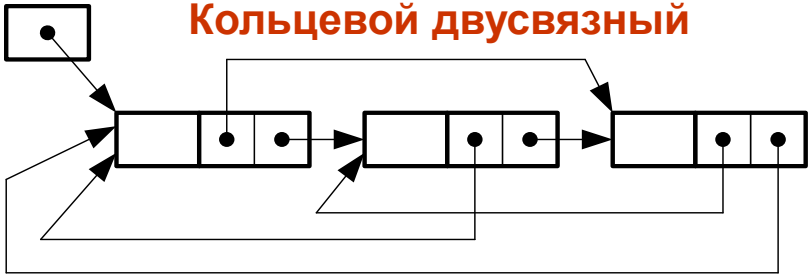
Линейный односвязный

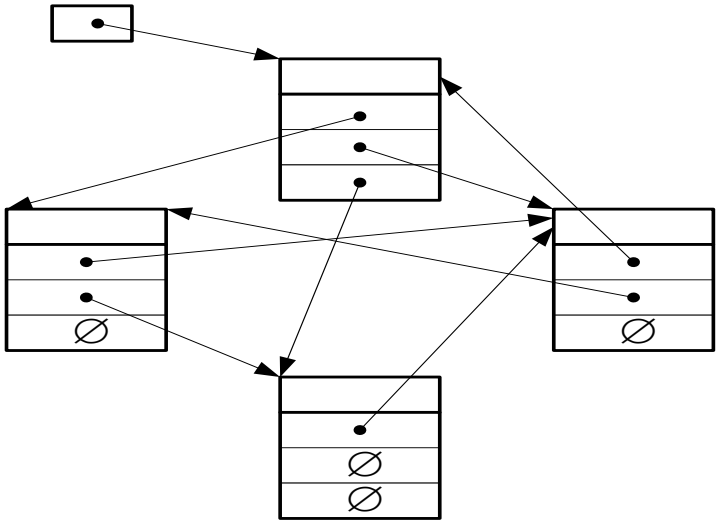
Кольцевой односвязный

Линейный двусвязный



Кольцевой двусвязный





Сетевой n-связный

Примеры описания элементов списка

Односвязный список:

```
struct el{
    char name[22];          // информационное поле 1
    char telefon[10];       // информационное поле 2
    el *p;                  // адресное поле
};
```

Двусвязный список:

```
struct element{
    char name[22];          // информационное поле 1
    char telefon[10];       // информационное поле 2
    element *prev;          // адресное поле «предыдущий»
    element *next;          // адресное поле «следующий»
};
```

5.3 Односвязные списки

Аналогично одномерным массивам реализуют последовательность элементов. Однако в отличие от одномерных массивов позволяют:

- работать **с произвольным количеством элементов**, добавляя и удаляя их по мере необходимости;
- осуществлять вставку и удаление записей, **не перемещая остальных элементов последовательности**;

но

- **не допускают прямого обращения к элементу по индексу;**
- **требуют больше памяти для размещения.**

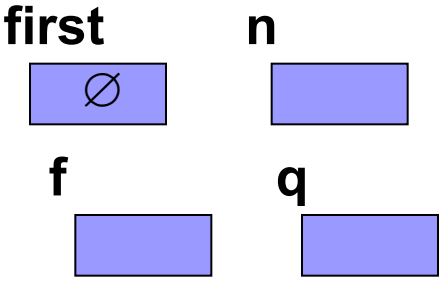
Объявление типа элемента и переменных

Описание типа элемента:

```
struct element {
    int num;        // число
    element *p;     // указатель на следующий элемент
};
```

Объявление переменной – указателя на начало списка и нескольких переменных-указателей в статической памяти:

```
element *first,  // адрес первого элемента
    *n,*f,*q;  // вспомогательные указатели
```



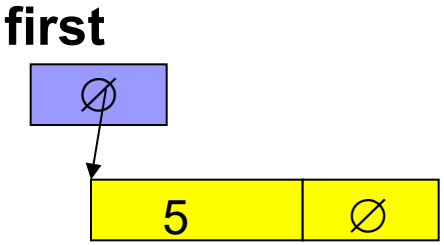
Исходное состояние – «список пуст»:

```
first = nullptr;
```

Добавление элементов к списку

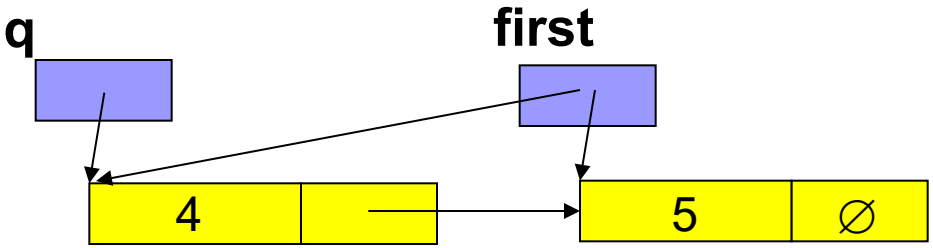
1 Добавление элемента к пустому списку:

```
first = new element;
first -> num = 5;
first -> p = nullptr;
```



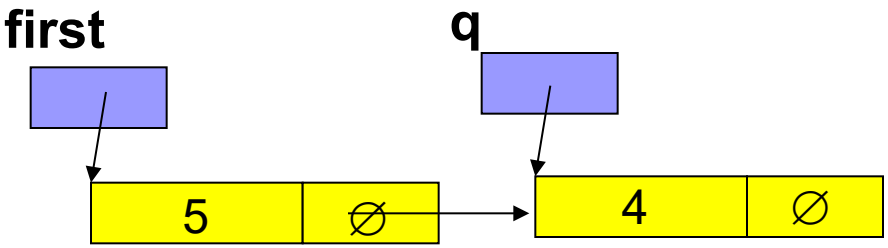
2 Добавление элемента перед первым (по типу стека):

```
q = new element;
q -> num = 4;
q -> p = first;
first = q;
```



3 Добавление элемента после первого (по типу очереди):

```
q = new element;
q -> num = 4;
q -> p = nullptr;
first -> p = q;
```

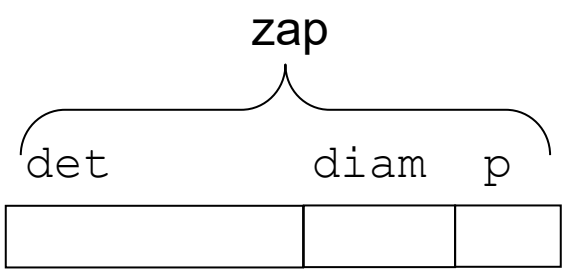


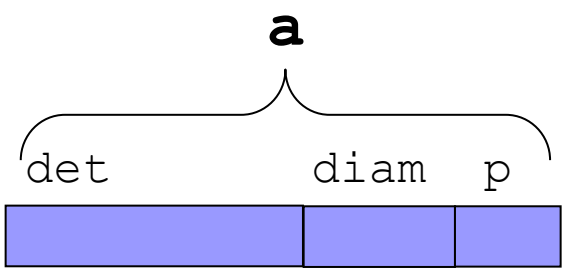
Пример. Стек записей

Задача. Разработать программу, которая строит список по типу стека из названий и диаметров деталей, и удаляет детали диаметром < 1.

Ex05_01

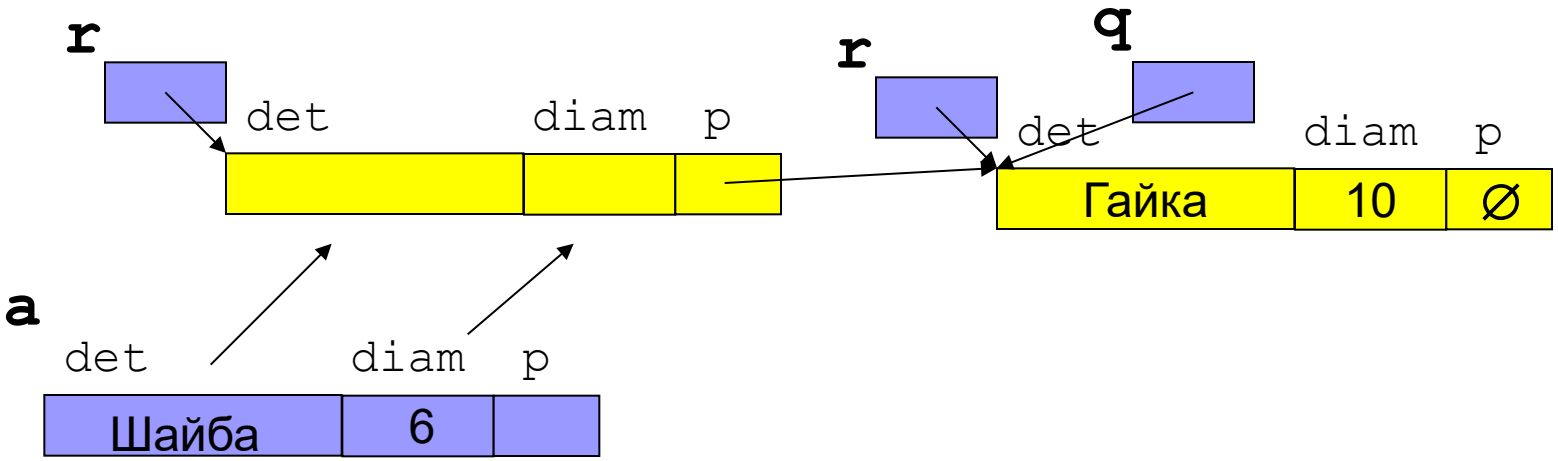
```
#include <iostream>
#include <string.h>
using namespace std;
struct zap {
   char det[10];
   float diam;
   zap *p;
};
int main()
{
     zap a,*r,*q,*f;
     r=new zap;
     r->p = nullptr;
     cin >> r->det >> r->diam;
```





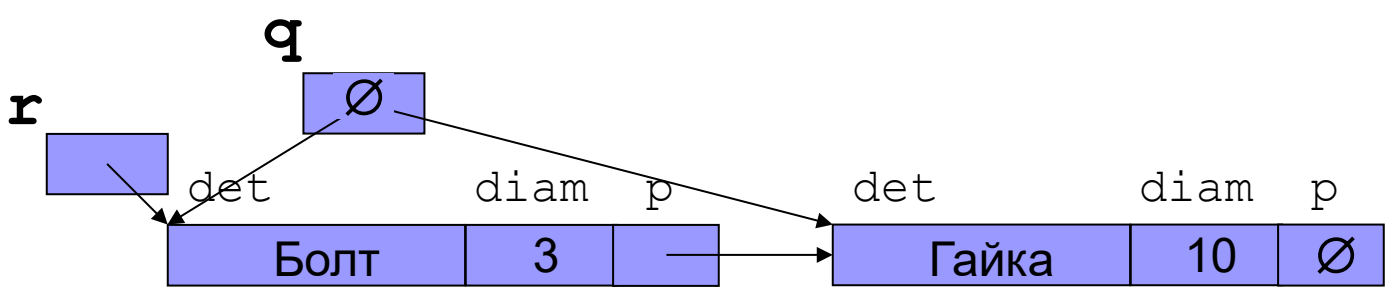
Стек записей. Создание списка по типу стека

```
while(cin >> a.det, strcmp(a.det, "end")!=0)
{
    cin >> a.diam;
    q=r;
    r=new zap;
    strcpy(r->det,a.det);
    r->diam=a.diam;
    r->p=q;
}
```

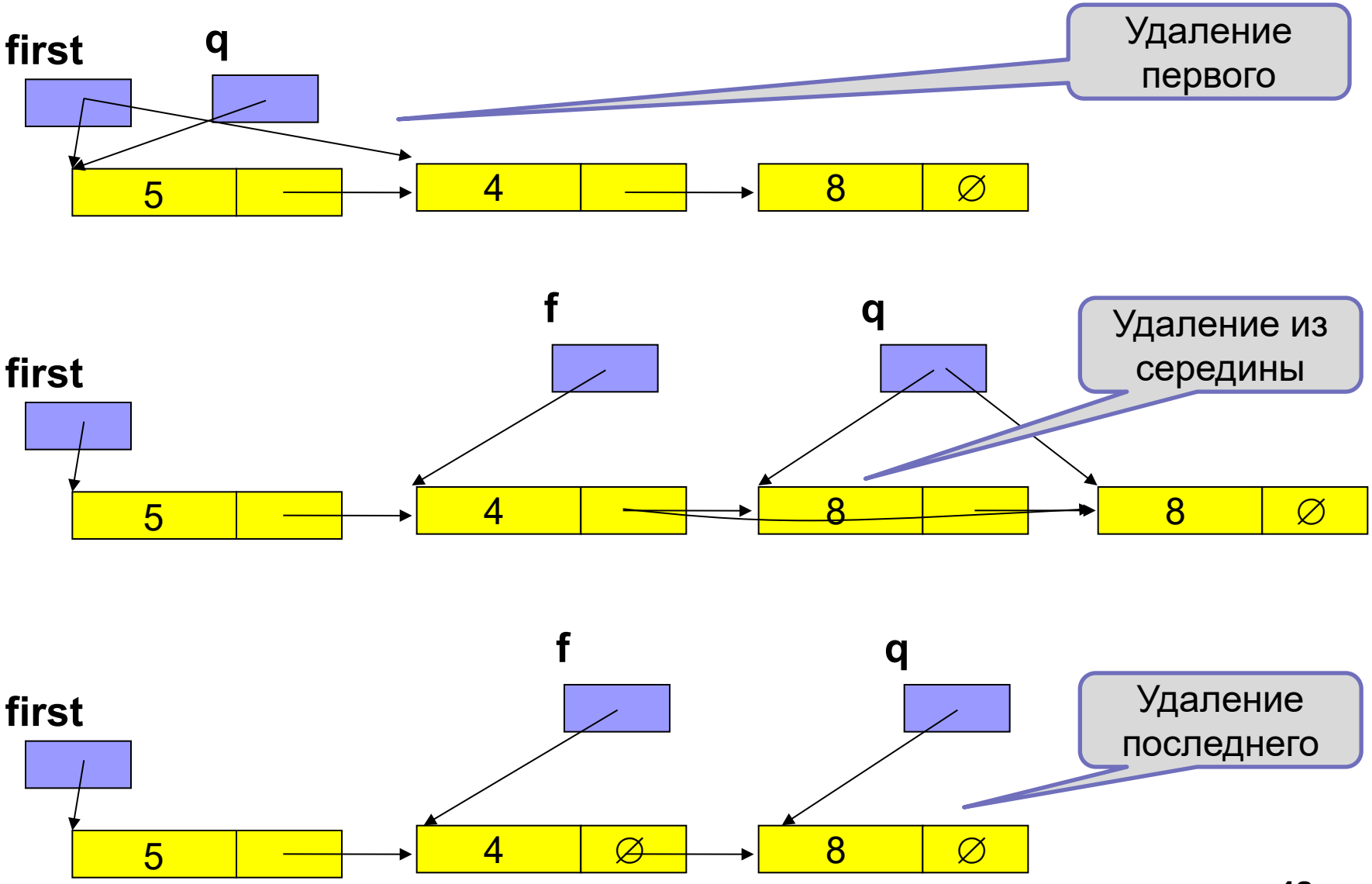


Стек записей. Вывод списка на экран

```
q=r;
cout << "List\n";
if(q == nullptr)
    cout << "No information.\n"; // список пуст
else
    do {
        cout << q->det << ' ' << q->diam << endl;
        q=q->p;
    }
    while (q!=nullptr);
```

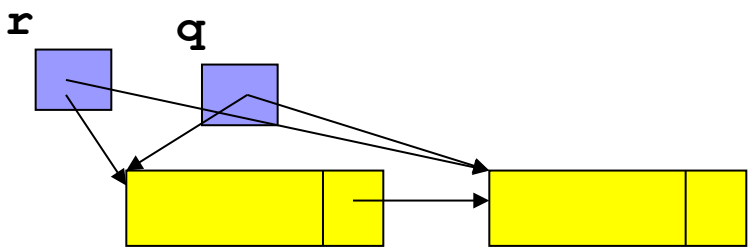


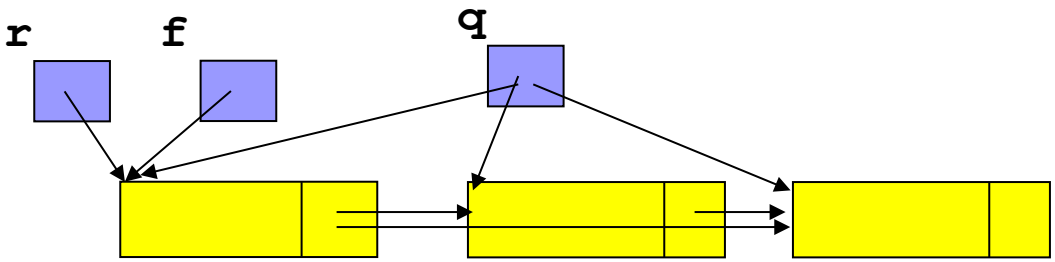
Варианты удаления элементов



Стек записей. Удаление деталей с диаметром < 1

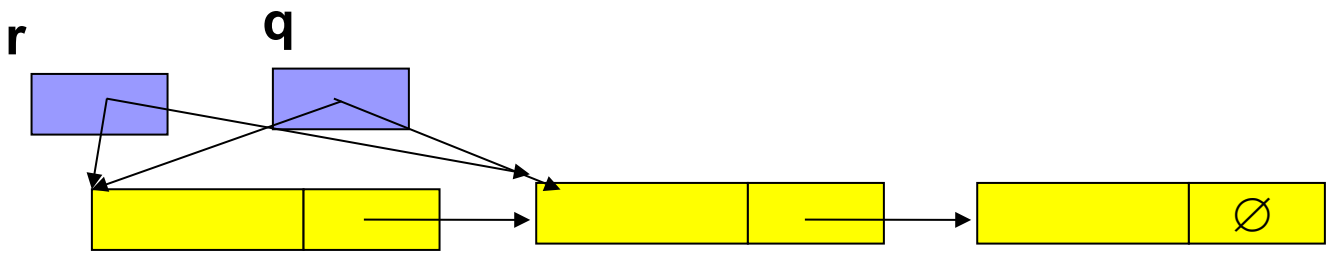
```
q=r;
do  {
   if(q->diam<1)
   {
      if(q==r) {
         r=r->p;
         delete q;
         q=r;
      }
      else  {
         q=q->p;
         delete f->p;
         f->p=q;
      }
   }
   else  {
      f=q;
      q=q->p;
   }
}
while(q!=nullptr);
```





Динамические структуры данных (5)

```
    q=r;
    cout << "Result\n";
    if(q == nullptr)
       cout << "No information.\n";
    else
       do {
            cout << q->det << q->diam << endl;
            r = q->p;
            delete q;        // освобождение памяти
            q = r;
       }
       while (q!=nullptr);
    return 0;
}
```

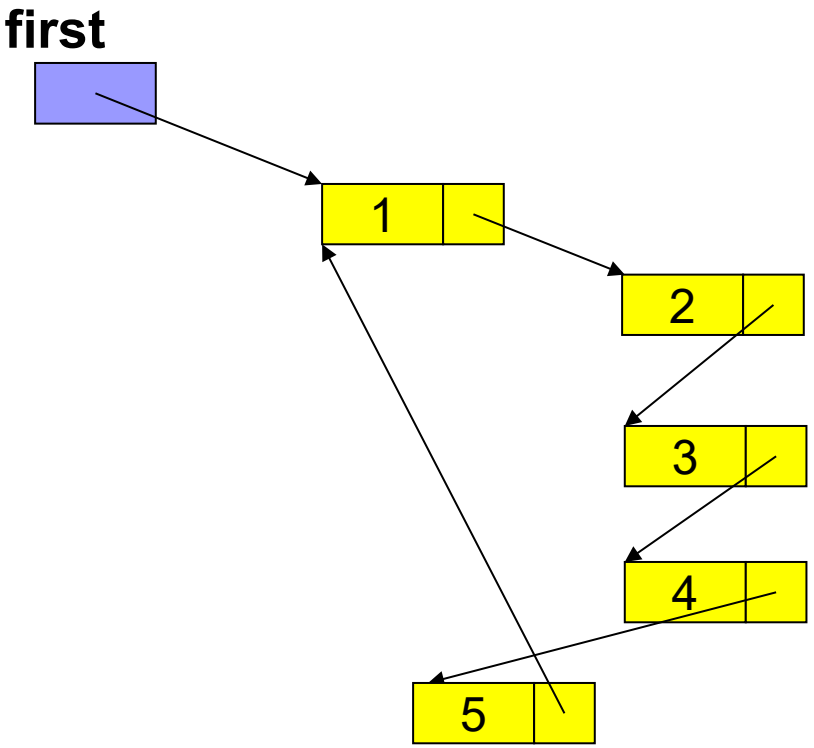


Кольцевой список

1 2 3 4 5

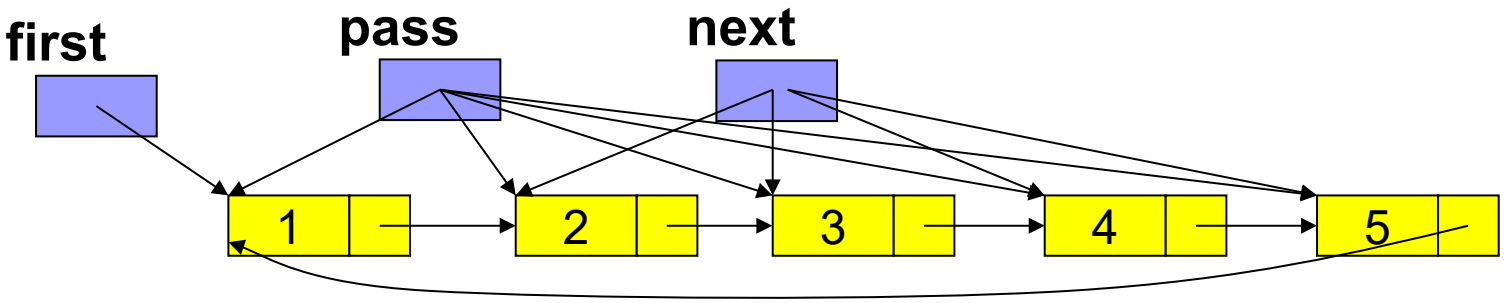
Ex05_02

```
#include <iostream>
using namespace std;

int play(int n,int m) {
    struct child {
        int name;
        child *p;
    };
    child *first,*next,*pass;
```



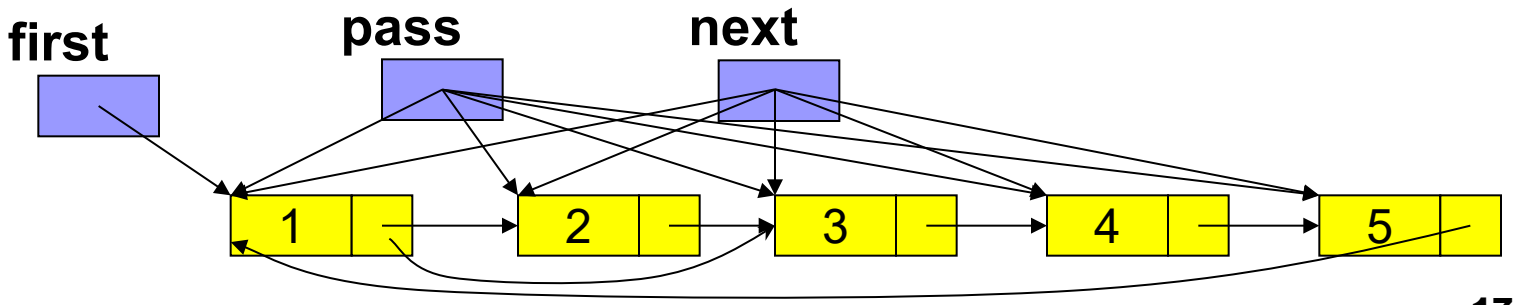
Создание списка

```
first = new child;  // создание первого элемента
first->name = 1;
pass = first;
for (int k=2;k<=n;k++) {
    next = new child; // создание остальных элементов
    next->name = k;
    pass->p = next;
    pass = next;
}
pass->p = first;  // замыкание круга
```



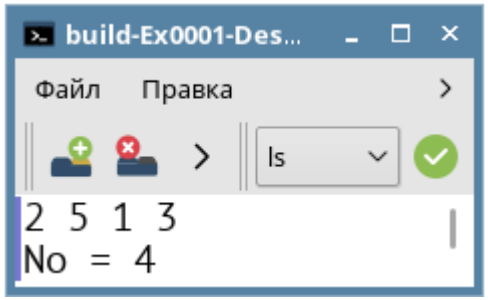
Проход по кольцу n-1 раз и удаление m-го элемента

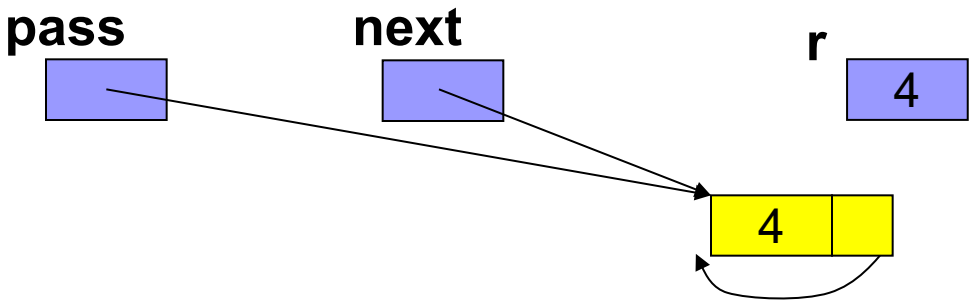
```
pass = first;
for (int k = n;k>1;k--) {
   for (int j=1;j<m;j++) {  // пропуск m-1 элемента
      next = pass;
      pass = pass->p;
   }
   cout << pass->name << ' ';
   next->p = pass->p;
   delete pass;             // удаление m-го элемента
   pass = next->p;
}
```



Возврат результата. Основная программа

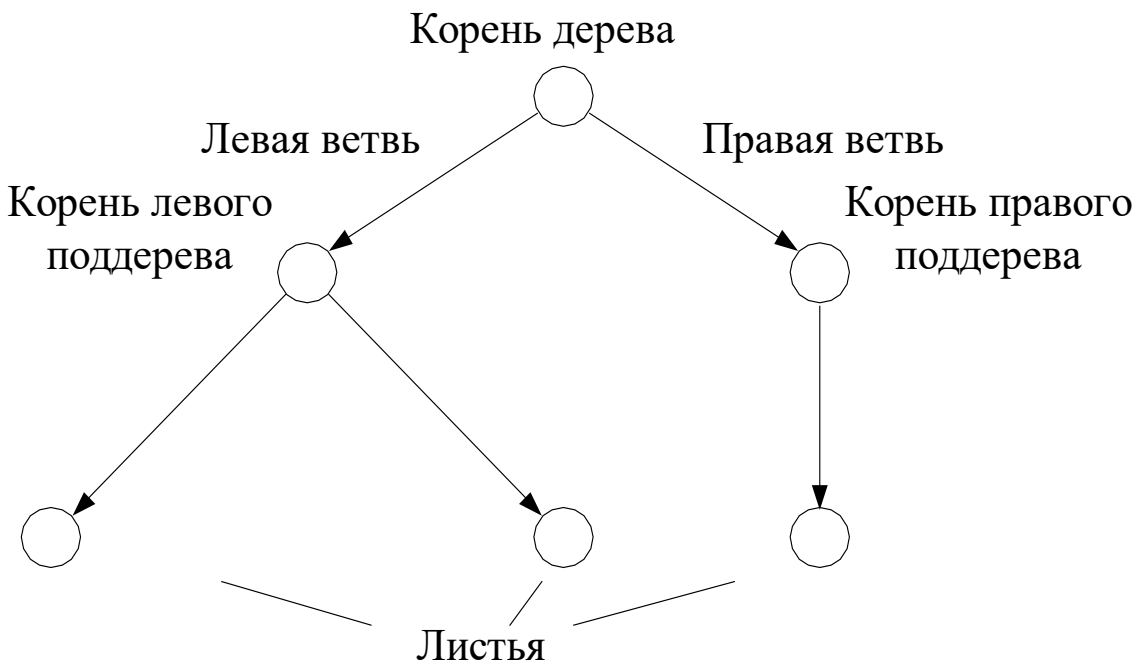
```
    int r = pass->name;
    cout << endl;
    delete pass;
    return r;
}

int main() {
    int y = play(5,7);
    cout << "No = " << y << endl;
    return 0;
}
```





6.5 Бинарные сортированные деревья

В математике ***бинарным деревом*** называют конечное множество вершин, которое либо пусто, либо состоит из корня и не более чем двух непересекающихся бинарных деревьев, называемых левым и правым поддеревьями данного корня.

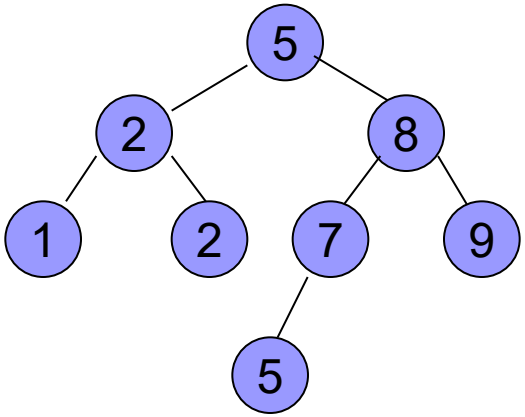


Вершины, из которых не выходит ни одной ветви, называют ***листьями***

Сортированные *бинарные деревья*, строятся по правилу: *ключевое поле левого поддерева должно содержать значение меньше, чем в корне, а ключевое поле правого поддерева – значение больше или равное значению в корне*.

Построение бинарного дерева

Рассмотрим последовательность целых чисел: {5, 2, 8, 7, 2, 9, 1, 5}



Пример. Разработать программу сортировки последовательности чисел с использованием бинарного дерева.

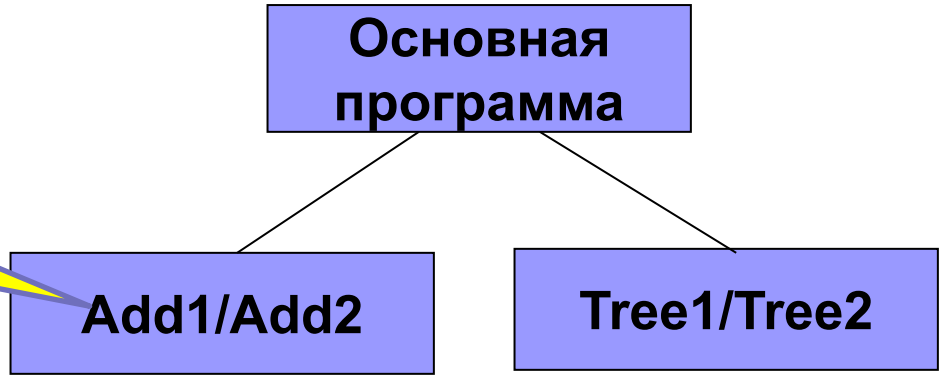
Описание элемента древовидного списка

```
#include <iostream>
using namespace std;

constexpr int lim = 100;

struct top {
    int value;
    top *left,*rigth;
};
```

Ex05_03

Выражение вычисляемое и подставляемое на этапе компиляции

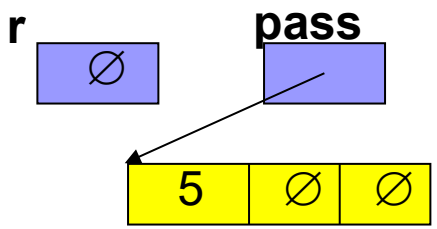
Схема структурная ПО



1 – нерекурсивный вариант;
2 – рекурсивный вариант -

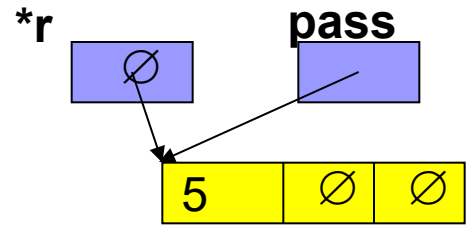
Основная программа

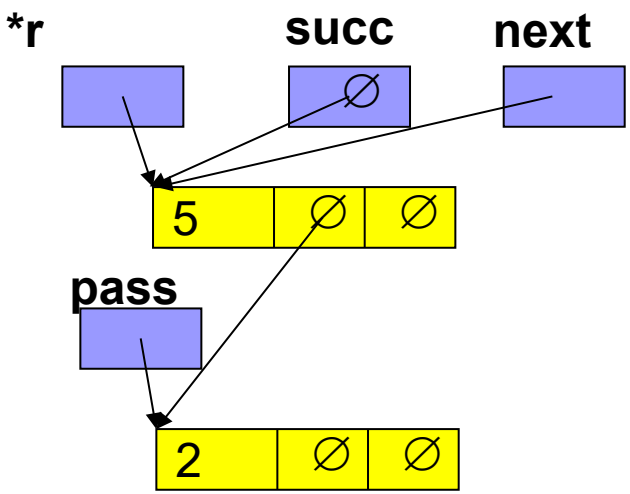
```
int main() {
    int next_number;
    top *r,*pass;
    cout << "Enter numbers (end - 1000):" << endl;
    r = nullptr;
    while (cin >> next_number,next_number != 1000) {
        pass = new top;
        pass->value = next_number;
        pass->left = pass->rigth = nullptr;
        Add1(&r,pass);
    }
    cout << "Result:" << endl;
    Tree1(r);
    return 0;
}
```



Нерекурсивная процедура построения дерева

```
void Add1(top **r,top *pass) {
    if (*r == nullptr) *r = pass;
    else {
                    top *next,*succ;
                    succ = *r;
                    while (succ != nullptr) {
                        next = succ;
                        if (pass->value<succ->value)
                            succ = succ->left;
                        else succ = succ->rigth;
                    }
                    if (pass->value<next->value)
                       next->left = pass;
                    else next->rigth = pass;
                }
            }
```





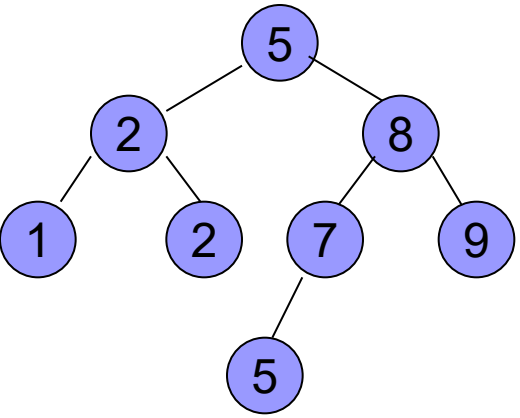
Рекурсивная процедура построения дерева

```
void Add2(top **r,top *pass) {
    if (*r == nullptr) *r = pass;
    else {
        if (pass->value < (*r)->value)
            Add2(&((*r)->left),pass);
        else Add2(&((*r)->rigth),pass);
    }
}
```

Нерекурсивная процедура обхода дерева

```
void Tree1(top *r) {
   top *pass;
   struct {
      int nom;
      top *adres[lim];
   } mem;
```

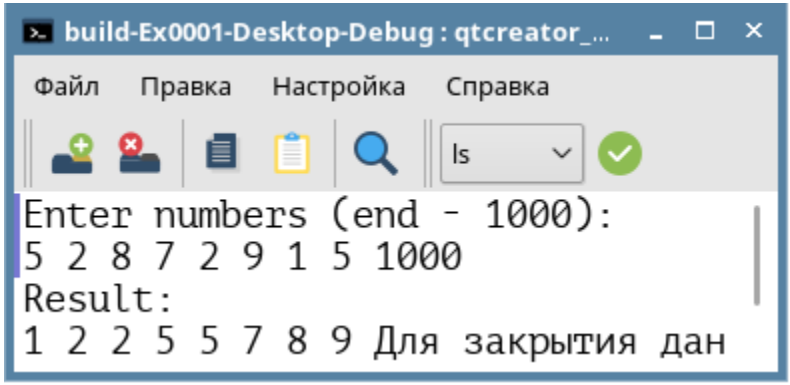
Нерекурсивная процедура обхода дерева (2)




```
    pass = r;
    mem.nom = -1;
    while (pass != nullptr || mem.nom != -1) {
          if (pass != nullptr) {
                if (mem.nom == lim) {
                      cout << "Error lim.\n";
                      exit(4);
                }
                mem.nom++;
                mem.adres[mem.nom] = pass;
                pass = pass->left;
          }
          else {
                pass = mem.adres[mem.nom];
                mem.nom--;
                cout << pass->value << ' ';
                pass = pass ->rigth;
          }
    }
}
```

Рекурсивная процедура обхода дерева

```
void Tree2(top *r) {
    if (r != nullptr) {
        Tree2(r->left);
        cout << r->value << ' ';
        Tree2(r->rigth);
    }
}
```



Примечание. При работе с деревом также необходимо предусмотреть процедуру освобождения памяти, занятой двоичным деревом!